

CNCO3003

Cloud Computing

Assignment 1

Authors:

21495959: Nazriel Al-Hafidz

Section 1 - Business Report

Business Name

ZBay

Description

ZBay is basically a small e-commerce startup that provides an online platform for local artisans to sell handmade products, such as jewelry, crafts and home decor. The company allows vendors to sign up, list their items and manage orders through a web-based dashboard. Customers can browse the catalogue and make purchases through the website, which I will set up to be accessible globally.

Cloud/In-House/Hybrid Infrastructure Justification

Since the business is planning to go global and its digital nature, ZBay is choosing to fully transition to a cloud computing system rather than relying on the traditional in-house infrastructure. Cloud computing offers scalability, flexibility and high availability which are all essential for a company like ZBay; an online marketplace expecting variable traffic and rapid growth.

Managing physical servers would introduce high upfront costs, require technical staff and limit our ability to scale on-demand. The AWS cloud model allows us to start small and grow as needed, while also minimising maintenance overhead and improving uptime.

AWS Cloud Services Used

Below is the list of features of Amazon's Web Services that I utilised to implement an efficient and productive cloud computing system for my ZBay business.

- **Amazon EC2:** Hosts the core web application that handles user interactions, product browsing, and order submissions. EC2 was chosen for its flexibility and integration with other AWS services, allowing ZBay to maintain full control over its backend environment.
- **Amazon VPC:** Enables secure deployment of ZBay's resources in isolated public and private subnets. This separation is essential for protecting sensitive data (like customer info in RDS) while exposing only the necessary public-facing web components.
- **Amazon RDS (MySQL):** I chose this for its reliability and ease of setup. This feature stores structured business data, including user accounts, products, and orders, with automated backups to protect against data loss.
- **Amazon EBS:** Provides ZBay scalable and persistent block storage attached to the EC2 instance. Used to store application data and logs. I opted for a 30 GiB volume to better reflect realistic storage needs for a production environment, with snapshot support for recovery.

- **Application Load Balancer (part of Elastic Load Balancing):** I used this to distribute incoming HTTP traffic across multiple EC2 instances, improving fault tolerance and ensuring users get fast, uninterrupted service even if one instance fails. I chose the ALB specifically for its ability to route requests at the application layer (port 80) and perform health checks on target instances.
- **Auto Scaling Group:** Automatically adds or removes EC2 instances based on CPU usage. This feature helps ZBay cost-effectively handle flash sales or marketing spikes without overpaying during quiet hours. Configured to maintain at least 2 instances at all times and scale up to 6 if traffic increases.
- **Amazon CloudWatch:** Monitors system metrics like CPU utilization, network traffic, and disk usage. I used CloudWatch to create a custom dashboard to monitor the health and performance of both EC2 and RDS resources. Also used it to trigger auto scaling actions based on CPU usage.
- **AWS IAM:** Centralised role and permission management. I used the predefined LabRole with broad permissions that allowed secure access to EC2, Lambda, and other services while adhering to AWS best practices for access control.

Estimated Monthly Cloud Expenses

I made a table based on the expected usage:

AWS Service	Description	Estimated Monthly Cost
EC2 (Auto Scaling)	1-2 t3a.medium (4GiB RAM, 2vCPU) instances with auto scaling.	\$60
RDS (MySQL Multi-AZ)	db.t3.small (2 vCPU, 2 GiB) with multi-AZ deployment.	\$45
EBS Storage	40 GB gp3 (applications + logs)	\$4.80
Load Balancer	Distributes traffic between instances (Application LB)	\$18
NAT Gateway + Data Transfer	Outbound internet access for private subnet.	\$25
Amazon CloudWatch	Logs, metrics, alarms and monitoring.	\$5
Total		\$157.80

So, from the table, we can say that the estimated monthly total to keep the cloud computing system running for ZBay is around **~\$160**.

Link to video presentation:

<https://youtu.be/fxPhxQBbFCI>

Script

Intro: Hi, my name is Nazriel, and this is my video presentation for the AWS Cloud Computing assignment. My fictional business is called ZBay, a small e-commerce startup that provides an online platform for local artisans to sell handmade products like jewelry, home decor, and organic personal care items.

Task 1: For IAM, I used the predefined LabRole provided by the AWS Academy Learner Lab environment. This role comes with 7 permission policies, including access to EC2, Lambda, EKS, and SSM. I didn't make any changes since it already has enough privileges for this assignment.

Task 2: In this task, I created a secure Virtual Private Cloud called ZBay-VPC with two availability zones for high availability. I set up two public subnets and two private subnets to prepare for future database integration. I attached an Internet Gateway and configured routing through a NAT Gateway to allow internet access for private resources. ZBay's multi-AZ VPC setup helps ensure high availability, and having NAT Gateways in each AZ prevents single points of failure for database backups and updates.

Next, I created a security group named ZBay-WebSG that allows incoming HTTP and SSH traffic. Finally, I launched an EC2 instance called ZBay-WebServer in the public subnet using Amazon Linux 2023. This instance is set up to accept web and SSH traffic and serves as the base for my web application.

Task 3: For Task 3, I developed and deployed an interactive web feature on my EC2 instance. I based it on the Amazon Q Developer lab activity, which involved creating an AWS Lambda function to validate email addresses. I integrated this Lambda function with a PHP-based web form hosted on my EC2 server. When users enter an email, the form calls the Lambda function through the AWS CLI, which returns whether the email is valid or not. The web feature is accessible through the EC2 public DNS and has been tested to ensure it responds correctly to different email inputs, demonstrating its functionality.

Task 4: To improve performance and prepare for increased web traffic, I upgraded my EC2 instance from t2.micro to t3a.medium, which provides 2 virtual CPUs and 4 GiB of RAM. This upgrade ensures smoother operation during peak demand while keeping costs under \$0.05 per hour.

I also enabled Termination Protection to avoid accidental deletion of the instance. For monitoring, I set up Amazon CloudWatch with detailed metrics on CPU usage, network traffic, and system health. I created a CloudWatch dashboard to visualize these metrics and included a screenshot in my report.

Task 5: For storage expansion, I created a new 30 GiB EBS volume more realistic for a production-like environment and attached it to my EC2 instance. After formatting it with the

ext4 filesystem, I mounted it to /mnt/zbay-data and verified that the additional storage was working. This shows how AWS allows scalable storage expansion on demand. I also took a snapshot of this volume to ensure I have a backup for future recovery.

Task 6:

Since ZBay is a small startup, I chose a Single-AZ Amazon RDS MySQL deployment to keep costs low and stay within my \$50 AWS credit budget. This setup is enough for now, and I can upgrade to Multi-AZ later as demand grows. To connect my EC2 web server to the RDS database, I installed the MariaDB client utilities on the instance, because the traditional MySQL client package wasn't available on Amazon Linux 2023. Specifically, I installed the mariadb1011-client-utils package, which corresponds to MariaDB version 10.1; a stable and recent release compatible with MySQL. I ran:

```
sudo dnf install mariadb1011-client-utils -y
```

This lets me run MySQL commands directly from the EC2 instance to manage the RDS MySQL database.

I launched the RDS instance in the same VPC as my EC2 server and created a custom security group that allows port 3306 traffic only from my web server's security group. From the EC2 instance, I connected to the RDS MySQL database using the MariaDB client, created a new database, and made a sample [products](#) table with test data to simulate an online store's product inventory. I verified that the EC2 instance can successfully connect and interact with the database. Screenshots of the RDS dashboard and SQL client connection are included to show full integration.

Task 7: For Task 7, I focused on increasing ZBay's availability and scalability by setting up Auto Scaling and a Load Balancer.

First, I created an Amazon Machine Image (AMI) from my existing web server instance. This AMI acts as a reusable template for launching new instances with the same configuration and software as my web server. It ensures that any new instance launched by the Auto Scaling Group (ASG) will be ready to serve ZBay's web application without extra setup.

Next, I created a Launch Template using the AMI, specifying instance type, security groups, and key pairs, so the ASG can launch instances consistently. Before creating the Load Balancer, I set up a Target Group with the protocol HTTP and port 80. This target group is where the load balancer will route incoming web traffic. For routing, I kept the default listener on port 80 and set the default action to forward traffic to this target group.

Then, I created an Application Load Balancer (ALB). This ALB distributes incoming HTTP requests evenly across all healthy instances registered in the target group, which improves fault tolerance and user experience. Finally, I created the Auto Scaling Group, linking it to the launch template and the target group behind the ALB. I set the desired and minimum capacity to 2 instances, ensuring ZBay's web application always

has at least two servers running to handle traffic. The maximum capacity was set to 6 to allow room for scaling up during higher demand.

To automatically scale the number of instances based on workload, I configured a Target Tracking Scaling Policy with a metric type of average CPU utilization, targeting 60%. This means if CPU usage across instances rises above 60%, the ASG will launch more instances to handle the load, and if it falls below, it will terminate instances to save costs. I added tags for organization, such as Name: ZBay-ASG and Environment: Production, to keep my AWS resources clearly labeled and easy to manage.

This setup ensures that ZBay’s web infrastructure can adapt dynamically to user traffic, providing high availability and scalability while optimizing costs; perfect for a growing e-commerce startup.

Screen Shots

IAM:

LabRole

Info

Delete

Summary

Edit

Creation date

May 20, 2025, 10:52 (UTC+08:00)

ARN

arn:aws:iam::622782162306:role/LabRole

Link to switch roles in console

<https://signin.aws.amazon.com/switchrole?roleName=LabRole&account=622782162306>

Instance profile ARN

arn:aws:iam::622782162306:instance-profile/LabInstanceProfile

Last activity

21 hours ago

Maximum session duration

1 hour

Permissions

Trust relationships

Tags (1)

Last Accessed

Revoke sessions

Permissions policies (7)

Info

Simulate

Remove

Add permissions

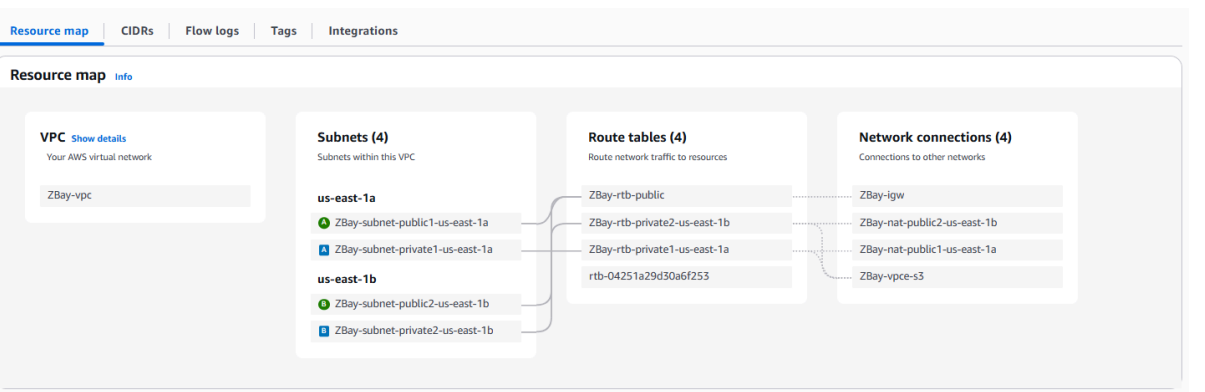
You can attach up to 10 managed policies.

Filter by Type

All types

Policy name	Type	Attached entities
AmazonEC2ContainerRegistryReadOnly	AWS managed	1
AmazonEKSClusterPolicy	AWS managed	1
AmazonEKSWorkerNodePolicy	AWS managed	1
AmazonSSMManagedInstanceCore	AWS managed	1
c162421a4166278110355058t1w622782162306-VocLabPolicy1-...	Customer managed	1
c162421a4166278110355058t1w622782162306-VocLabPolicy2-...	Customer managed	1
c162421a4166278110355058t1w622782162306-VocLabPolicy3-...	Customer managed	1

VPC:



Security group:

sg-02a129b206562742a - ZBay-WebSG

Actions

Details

Security group name

ZBay-WebSG

Security group ID

sg-02a129b206562742a

Description

Allows HTTP and SSH access

VPC ID

vpc-0add8df7a0ad1fd2b

Owner

622782162306

Inbound rules count

4 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing - new

VPC associations - new

Tags

Inbound rules (4)

Manage tags

Edit inbound rules

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sgr-05a93276c557f4a5b	IPv4	SSH	TCP	22	0.0.0.0/0
☐	-	sgr-00b62acf8f68e88c4	IPv4	HTTP	TCP	80	0.0.0.0/0
☐	-	sgr-0d052cde2f4362bf8	-	MYSQL/Aurora	TCP	3306	sg-02a129b206562742a
☐	-	sgr-0ec50fca06d6434b9	IPv4	Custom TCP	TCP	5000	0.0.0.0/0

sg-02a129b206562742a - ZBay-WebSG

Actions

Details

Security group name

ZBay-WebSG

Security group ID

sg-02a129b206562742a

Description

Allows HTTP and SSH access

VPC ID

vpc-0add8df7a0ad1fd2b

Owner

622782162306

Inbound rules count

4 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing - new

VPC associations - new

Tags

Outbound rules (1)

Manage tags

Edit outbound rules

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination
☐	-	sgr-089d268e23ce9fdb0	IPv4	All traffic	All	All	0.0.0.0/0

EC2 Instance

Instance summary for i-0c12b5229b19b1461 (ZBay-WebServer) Info

Connect

Instance state

Actions

Updated less than a minute ago

Instance ID

i-0c12b5229b19b1461

IPv6 address

-

Hostname type

IP name: ip-10-0-0-215.ec2.internal

Answer private resource DNS name

-

Auto-assigned IP address

54.81.87.183 [Public IP]

IAM Role

LabRole

IMDSv2

Required

Operator

-

Public IPv4 address

54.81.87.183 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-0-0-215.ec2.internal

Instance type

t3a.medium

VPC ID

vpc-0add8df7a0ad1fd2b (ZBay-vpc)

Subnet ID

subnet-054fd9235185fb607 (ZBay-subnet-public1-us-east-1a)

Instance ARN

arn:aws:ec2:us-east-1:622782162306:instance/i-0c12b5229b19b1461

Private IPv4 addresses

10.0.0.215

Public DNS

ec2-54-81-87-183.compute-1.amazonaws.com | open address

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

-

Managed

false

▼ Security details

IAM Role

LabRole

Security groups

sg-02a129b206562742a (ZBay-WebSG)

Owner ID

622782162306

Launch time

Sat May 24 2025 16:35:40 GMT+0800 (Australian Western Standard Time)

▼ Inbound rules

Filter rules

< 1 >

Name	Security group rule ID	Port range	Protocol	Source	Security groups	Description
-	sgr-05a93276c557f4a5b	22	TCP	0.0.0.0/0	ZBay-WebSG	Permit remote i
-	sgr-00b62acf8f68e88c4	80	TCP	0.0.0.0/0	ZBay-WebSG	Permit web req
-	sgr-0d052cde2f4362bf8	3306	TCP	sg-02a129b206562742a	ZBay-WebSG	-
-	sgr-0ec50fca06d6434b9	5000	TCP	0.0.0.0/0	ZBay-WebSG	Email validator

▼ Outbound rules

Filter rules

< 1 >

Name	Security group rule ID	Port range	Protocol	Destination	Security groups	Description
-	sgr-089d268e23ce9fdb0	All	All	0.0.0.0/0	ZBay-WebSG	-

Web App Feature:

GNU nano 8.3 /var/www/html/validate.php

<html>
<body>
<h2>Email Validator</h2>
<form method="post">
 Email: <input type="text" name="email">
 <input type="submit" value="Validate">
</form>

<?php
if (\$_SERVER["REQUEST_METHOD"] == "POST") {
 \$email = \$_POST['email'];
 \$payload = json_encode(["email" => \$email]);
 \$command = "aws lambda invoke --function-name myFunction --payload '\$payload' --cli-binary-format raw-in-base64-out /tmp/response.json";
 shell_exec(\$command);
 \$responseJson = file_get_contents("/tmp/response.json");
 \$response = json_decode(\$responseJson, true);
 \$message = json_decode(\$response["body"], true); // Get the string inside the "body"
 echo "<h3>Result:</h3>";
 echo "<p>\$message</p>";
}
?>
</body>
</html>

Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was

i-0c12b5229b19b1461 (ZBay-WebServer)
PublicIPs: 54.81.103.198 PrivateIPs: 10.0.0.215

← ↻ 🏠 ⚠ Not secure ec2-54-81-87-183.compute-1.amazonaws.com/validate.php

🔖 Import favorites | 📧 Inbox (155) - nazrul...

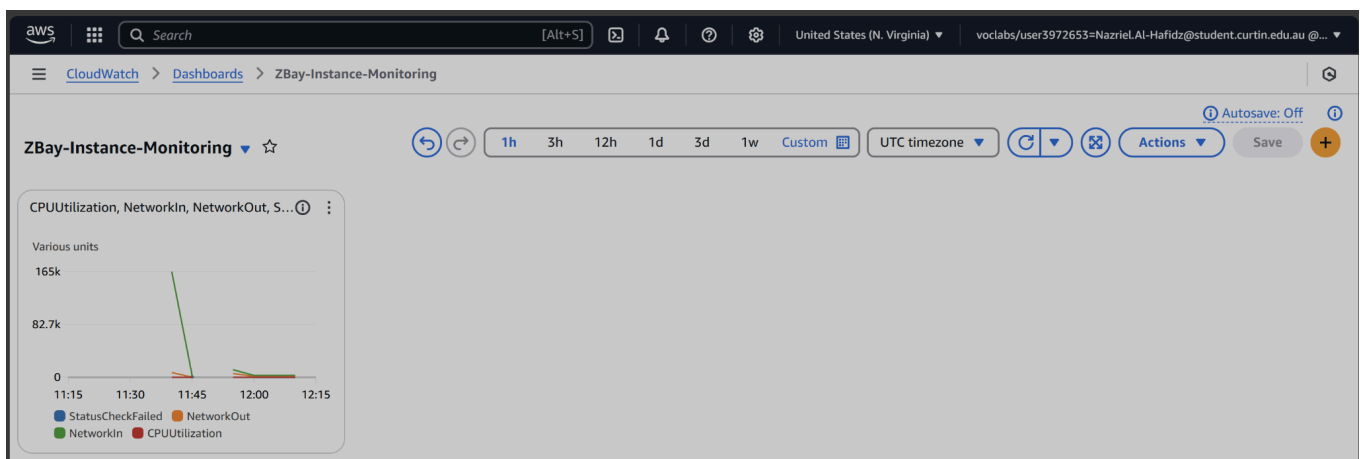
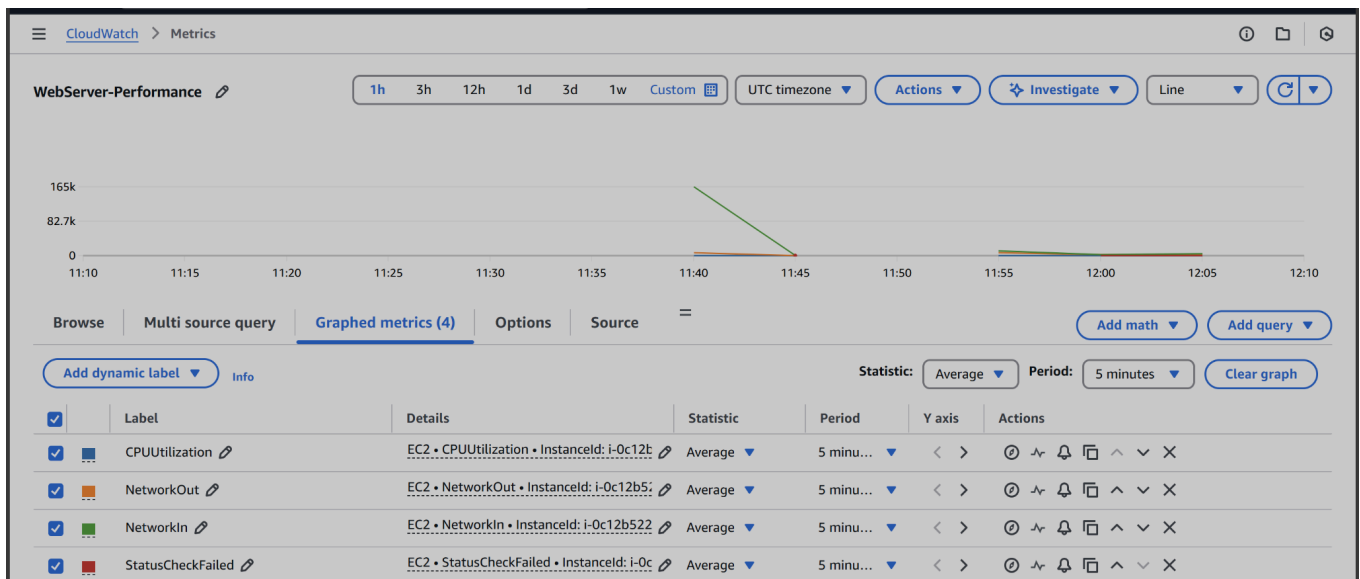
Email Validator

Email:

Result:

Email address is valid!

CloudWatch Monitoring



Volume Mounting

```
Last login: Wed May 21 12:47:48 2025 from 18.206.107.29
[ec2-user@ip-10-0-0-215 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0   8G  0 disk
├─nvme0n1p1 259:1    0   8G  0 part /
├─nvme0n1p127 259:2    0   1M  0 part
└─nvme0n1p128 259:3    0  10M  0 part /boot/efi
nvme1n1     259:4    0  30G  0 disk /mnt/zbay_data
```

```

nvme0n1 253.1G 0 253.1G 0 disk /mnt/zbay_data
[ec2-user@ip-10-0-0-215 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0  4.0M   0% /dev
tmpfs           1.9G   0  1.9G   0% /dev/shm
tmpfs           768M  8.5M  760M   2% /run
/dev/nvme0n1p1  8.0G  1.7G  6.3G  21% /
tmpfs           1.9G   0  1.9G   0% /tmp
/dev/nvme0n1p128 10M   1.3M  8.7M  13% /boot/efi
/dev/nvme1n1    30G   24K   28G   1% /mnt/zbay_data
tmpfs          384M   0  384M   0% /run/user/1000

```

Log-in details to connect to RDS

Username: admin

Password: ZBayAdmin123

MySQL through RDS Endpoint:

```

MySQL [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.036 sec)

MySQL [(none)]> create database zbaydb;
Query OK, 1 row affected (0.004 sec)

MySQL [(none)]> use zbaydb;
Database changed

```

```

MySQL [zbaydb]> create table products (
  -> id INT AUTO_INCREMENT PRIMARY KEY,
  -> name VARCHAR(100),
  -> price DECIMAL(10,2),
  -> stock INT
  -> );
Query OK, 0 rows affected (0.049 sec)

MySQL [zbaydb]> show tables;
+-----+
| Tables_in_zbaydb |
+-----+
| products |
+-----+
1 row in set (0.002 sec)

```

```
MySQL [zbaydb]> desc products;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
price	decimal(10,2)	YES		NULL	
stock	int	YES		NULL	

```
4 rows in set (0.011 sec)
```

```
MySQL [zbaydb]> insert into products (name, price, stock) values
-> ('Beaded Charm Bracelet', 9.99, 30),
-> ('Handcrafted Ceramic Mug', 7.50, 27),
-> ('Knitted Wool Scarf', 14.99, 34),
-> ('Wooden Jewelry Box', 12.49, 18),
-> ('Small Batch Natural Lip Balm', 21.99, 42);
```

```
Query OK, 5 rows affected (0.004 sec)
```

```
Records: 5  Duplicates: 0  Warnings: 0
```

```
MySQL [zbaydb]> select*from products;
```

id	name	price	stock
1	Beaded Charm Bracelet	9.99	30
2	Handcrafted Ceramic Mug	7.50	27
3	Knitted Wool Scarf	14.99	34
4	Wooden Jewelry Box	12.49	18
5	Small Batch Natural Lip Balm	21.99	42

```
5 rows in set (0.001 sec)
```

Launch Template

ZBay-LaunchTemplate (lt-00299ac20356baaf1)

ActionsDelete template

Launch template details

Launch template ID
lt-00299ac20356baaf1

Launch template name
ZBay-LaunchTemplate

Default version
1

Owner
arn:aws:sts::622782162306:assumed-role/voclabs/user3972653=Nazriel.AL-Hafidz@student.curtin.edu.au

Details

Versions

Template tags

Launch template version details

ActionsDelete template version

Version
1 (Default)

Description
-

Date created
2025-05-23T08:19:45.000Z

Created by
arn:aws:sts::622782162306:assumed-role/voclabs/user3972653=Nazriel.AL-Hafidz@student.curtin.edu.au

Instance details

Storage

Resource tags

Network interfaces

Advanced details

AMI ID
ami-01a781ffec6c7346a

Instance type
t3a.medium

Availability Zone
-

Key pair name
zbay-key

Security groups
-

Security group IDs
sg-02a129b206562742a

Review

Review the load balancer configurations and make changes if needed. After you finish reviewing the configurations, choose Create load balancer.

Summary

Review and confirm your configurations. Estimate cost

Basic configuration Edit

Name: ZBay-ELB

Scheme: Internet-facing

IP address type: IPv4

Network mapping Edit

VPC: vpc-0add8df7a0ad1fd2b

Public IPv4 IPAM pool: -

Availability Zones and subnets:

- us-east-1a
subnet-054fd9235185fb607
ZBay-subnet-public1-us-east-1a
- us-east-1b
subnet-06aa89d67407cf1ba
ZBay-subnet-public2-us-east-1b

Security groups Edit

ZBay-WebSG

sg-02a129b206562742a

Listeners and routing Edit

HTTP:80 | Target group: ZBay-TargetGroup

Service integrations Edit

Amazon CloudFront + AWS Web Application Firewall (WAF): -

AWS WAF: -

AWS Global Accelerator: -

Tags Edit

-

Attributes

Certain default attributes will be applied to your load balancer. You can view and edit them after creating the load balancer.

Full Lambda function as text:

```
import json

def lambda_handler(event, context):
    # TODO implement
    #Get the email address from the event
    email = event['email']
    #Validate the email address
    if validate_email(email):
        return {
            'statusCode': 200,
            'body': json.dumps('Email address is valid!')
        }
    #Return an error message
    else:
        return {
            'statusCode': 400,
            'body': json.dumps('Email address is not valid!')
        }

#Function that validates an email address using regex
def validate_email(email):
    import re
    regex =
r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
    if(re.fullmatch(regex, email)):
        return True
    else:
        return False
```